

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANA SANGAMA, BELAGAVI - 590018, KARNATAKA



Lab Manual On

**“MICROCONTROLLERS & EMBEDDED SYSTEMS
LABORATORY” BCO601**

VI Semester of
Bachelor of Engineering in Computer Science and Engineering (Aiml) of
Visvesvaraya Technological University, Belagavi

Mr.L.Sreenivasaperumal

Assistant Professor

Dept. of Computer Science and Engineering
Artificial Intelligence and Machine Learning

AMC Engineering College



Department of Computer Science and Engineering

Artificial Intelligence and Machine Learning

AMC ENGINEERING COLLEGE

18 Km, Bannerghatta Road, Bangalore - 560083

2024-25



COLLEGE VISION

“To be a leader in imparting value Based Technical Education and Research for the benefit of society”

COLLEGE MISSION

- **To provide state of the art Infrastructure facilities.**
- **To implement modern Pedagogical Methods in delivering the Academic Programs with Experienced and Committed Faculty.**
- **To create a Vibrant Ambience that promotes Learning, Research, Invention and Innovation.**
- **To undertake Skill Development Programmes for Academic Institutions and Industries.**
- **To Enhance Industry Institute Interaction through Collaborative Research and Consultancy.**
- **To Relentlessly Pursue Professional Excellence with Ethical and Moral Values.**

Department of CSE-AIML

DEPARTMENT VISION

Be a premier department in the field of Computer Science & Engineering - Artificial Intelligence & Machine Learning (CSE - AIML) to meet the technological challenges of the society.

DEPARTMENT MISSION

MD1: To facilitate state of the art infrastructure by providing exposure to the latest AI tools used in a technically competent society.

MD2: To strive for academic excellence through research in Artificial Intelligence & Machine Learning with creative teaching-learning pedagogy.

MD3: To establish Industry Institute Interaction and make students ready for the Industrial environment.

MD4: To transform students into entrepreneurial, technically competent, socially responsible, and ethical AI professional.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

<i>PEO 1</i>	<i>Graduates possess advanced knowledge of Computer Science & Engineering - Artificial Intelligence & Machine Learning and excel in leadership roles to serve the society.</i>
<i>PEO 2</i>	<i>Graduates of the program will apply Artificial Intelligence enabled Computer Engineering tools in core technologies for improving knowledge in the Interdisciplinary Research and Entrepreneurs.</i>
<i>PEO 3</i>	<i>Graduates adapt Value-Based Proficiency in solving Real Time problems.</i>

PROGRAM SPECIFIC OUTCOMES (PSOs)

<i>PSO-1</i>	<i>Professional Skills: Ability of applying the Artificial Intelligence & Machine Learning knowledge, Computing Concepts, Data Structure, Computer Hardware, Computer Networks and Suitable Algorithm.</i>
<i>PSO-2</i>	<i>Software Skills: Ability to build Artificial Intelligence enabled Software Engineering System with Development Life Cycle by using analytical knowledge in Computer Science & Engineering and applying modern methodologies.</i>

PROGRAM OUTCOMES (POS)

<i>P01</i>	<i>Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.</i>
<i>P02</i>	<i>Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.</i>
<i>P03</i>	<i>Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.</i>
<i>P04</i>	<i>Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions</i>
<i>P05</i>	<i>Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and</i>

	<i>modeling to complex engineering activities with an understanding of the limitations.</i>
<i>P06</i>	<i>The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.</i>
<i>P07</i>	<i>Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.</i>
<i>P08</i>	<i>Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.</i>
<i>P09</i>	<i>Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.</i>
<i>P010</i>	<i>Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.</i>
<i>P011</i>	<i>Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.</i>
<i>P012</i>	<i>Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.</i>



AMC ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING -AIML

Course Details

Microcontrollers & Embedded Systems Subject Code: BCO601

Course objectives:

- Understand the architectural features and instruction set of 32-bit ARM microcontrollers.
- Apply instructions of assembly language for programming ARM.
- Interpret the basic hardware components and their selection method based on the characteristics and attributes of an embedded system.
- Explain the need of real time operating system for embedded system applications.
- Develop/test/Conduct the experiments on an ARM7TDMI/LPC2148 evaluation board using

Embedded 'C' and Keil Vision tool/Compiler **CIE for the practical component of the IPCC**

- **15 marks** for the conduction of the experiment and preparation of laboratory record, and **10 marks** for the test to be conducted after the completion of all the laboratory sessions.
- On completion of every experiment/program in the laboratory, the students shall be evaluated including viva-voce and marks shall be awarded on the same day.
- The CIE marks awarded in the case of the Practical component shall be based on the continuous evaluation of the laboratory report. Each experiment report can be evaluated for 10 marks. Marks of all experiments' write-ups are added and scaled down to **15 marks**.
- The laboratory test (**duration 02/03 hours**) after completion of all the experiments shall be conducted for 50 marks and scaled down to **10 marks**.
- Scaled-down marks of write-up evaluations and tests added will be CIE marks for the laboratory component of IPCC for **25 marks**.

The student has to secure 40% of 25 marks to qualify in the CIE of the practical component of the IPCC.

Experiments

Conduct the following experiments by writing programs using ARM7TDMI/LPC2148 using an evaluation board/simulator/evaluation version of Embedded 'C' & Keil Uvision-4 tool/compiler. and the required software tool.

S.NO	experiments
1	Develop a program to multiply two 16-bit binary numbers.
2	Write a program to find the sum of first 10 integer numbers.
3	a. Write a program to find factorial of a number.
4	Write a program to add an array of 16-bit numbers and store the 32-bit result in internal RAM
5	Write a program to find the square of a number (1 to 10) using look-up table.
6	Write a program to find the largest/smallest number in an array of 32 numbers.
7	Display “Hello World” message using Internal UART.
8	Interface a Stepper motor and rotate it in clockwise and anti-clockwise direction
9	Display the Hex digits 0 to F on a 7-segment LED interface, with an appropriate delay in between
10	Interface a 4x4 keyboard and display the key code on an LCD.

Course Outcome

CO1	Explain the architectural features and instructions of ARM microcontroller
CO2	Apply the knowledge gained for Programming ARM for different applications.
CO3	Demonstrate Interfacing of external devices and I/O with ARM microcontroller.
CO4	Interpret the basic hardware components and their selection method based on the characteristics and attributes of an embedded system.
CO5	Develop the hardware /software co-design and firmware design approaches.

TABLE OF CONTENTS

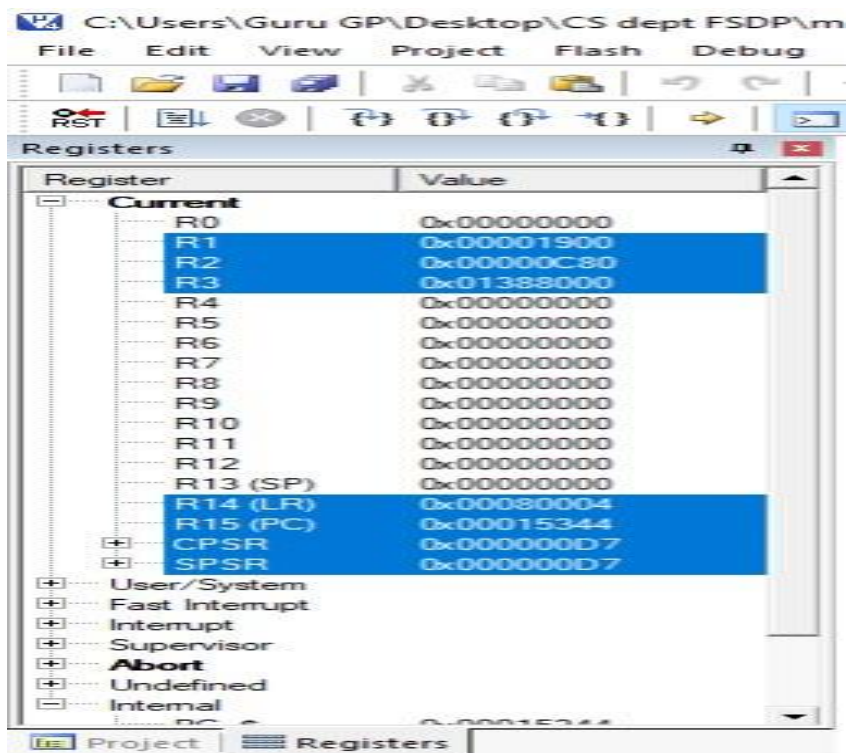
INDEX

S.NO	EXPERIMENTS	PAGE NO
1.	Develop a program to multiply two 16-bit binary numbers.	4
2.	Write a program to find the sum of first 10 integer numbers.	6
3.	Write a program to find factorial of a number.	9
4.	Write a program to add an array of 16-bit numbers and store the 32-bit result in internal RAM	12
5.	Write a program to find the square of a number (1 to 10) using look-up table.	15
6.	Write a program to find the largest/smallest number in an array of 32 numbers.	17
7.	Display “Hello World” message using Internal UART.	18
8.	Interface a Stepper motor and rotate it in clockwise and anti-clockwise direction	20
9.	Display the Hex digits 0 to F on a 7-segment LED interface, with an appropriate delay in between	21
10	Interface a 4x4 keyboard and display the key code on an LCD.	12
II	SAMPLE VIVA QUESTIONS AND ANSWERS	26-29

Program 1:

Write a program to multiply two 16-bit binary numbers.

```
AREA MULTIPLY , CODE, READONLY
ENTRY                               ; Mark first instruction to execute
START
    MOV r1,#6400                    ; STORE FIRST NUMBER IN R0
    MOV r2,#3200                    ; STORE SECOND NUMBER IN R1
    MUL r3,r1,r2                    ; MULTIPLICATION
    NOP
    NOP
    NOP
BACK B BACK
END                                 ;Mark end of file
```



Program 2:

Write a program to find the sum of first 10 integer numbers.

```
AREA SUM, CODE, READONLY
```

```
ENTRY
```

```
MOV R1, #10          ; load 10 to register
```

```
MOV R2, #0           ; empty the register to store result
```

```
loop
```

```
ADD R2, R2, R1        ; add the content of R1 with result at R2
```

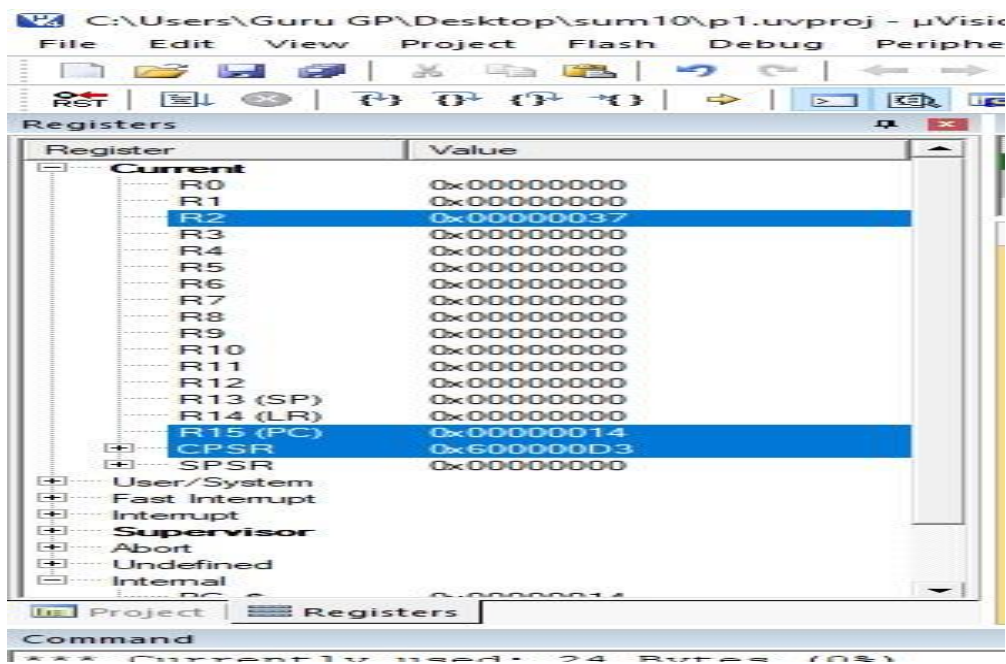
```
SUBS R1, #0x01        ; Decreament R1 by 1
```

```
BNE loop              ; repeat till r1 goes 0
```

```
back B back            ; jumps back to C code
```

```
END
```

RESULT:



Program 3

Write a program to find factorial of a number.

AREA FACTORIAL , CODE, READONLY

ENTRY ;Mark first instruction to execute

START

```
MOV r0, #5 ; STORE FACTORIAL NUMBER IN R0
```

```
MOV r1,r0           ; MOVE THE SAME NUMBER IN R1
```

FACT SUBS r1, r1, #1 ; SUBTRACTION

```
CMP r1, #1      ; COMPARISON
```

BEQ STOP

```
MUL r3,r0,r1    ; MULTIPLICATION
```

```
MOV r0,r3           ; Result
```

[illegible]

STOP

NOP

NOP

NOP

```
END                                ;Mark end of file
```

C:\Users\Guru GP\Desktop\CS dept FSDP\factoria

File Edit View Project Flash Debug Perip



Registers

Register	Value
Current	
R0	0x00000078
R1	0x00000001
R2	0x00000000
R3	0x00000078
R4	0x00000000
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00080004
R15 (PC)	0x000595BC
CPSR	0x600000D7
SPSR	0x600000D7
+	User/System
+	Fast Interrupt
+	Interrupt
+	Supervisor
+	Abort
+	Undefined
-	Internal

Program 4:

Write a program to add an array of 16 bit numbers and store the 32 bit result in internal RAM.

```
                AREA  ADDITION , CODE, READONLY

ENTRY          ; Mark first instruction to execute

START

    MOV R5,#6           ; INTIALISE COUNTER TO 6(i.e. N=6)

    MOV R0,#0           ; INTIALISE SUM TO ZERO

    LDR R1,=VALUE1      ; LOADS THE ADDRESS OF FIRST VALUE

LOOP

    LDRH R3,[R1],#02     ; READ 16 BIT DATA

    ADD R0,R0,R3         ; ADD R0=R0+R3

    SUBS R5,R5,#1        ; DECREMENT COUNTER

    CMP R5,#0

    BNE LOOP           ; LOOK BACK TILL ARRAY ENDS

    LDR R4,=RESULT      ; LOADS THE ADDRESS OF RESULT

    STR R0,[R4]         ; STORES THE RESULT IN MEMORY

    JMP  B JMP

VALUE1 DCW  0X1111, 0X2222, 0X3333, 0XAAAA, 0XBBBB, 0XCCCC    ; ARRAY OF 16 BIT NUMBERS (N=6)

                AREA  DATA2,DATA,READWRITE                    ; TO STORE RESULT

RESULT DCD 0X0

    END                ; Mark end of file
```

Result:

Register	Value
R0	0x00029997
R1	0x00000038
R2	0x00000000
R3	0x0000CCCC
R4	0x40000000
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000000
R15 (PC)	0x00000028
CPSR	0x600000D3
SPSR	0x00000000

User/System

Fast Interrupt

Project Registers

Program 5

Write a program to find the square of a number (1 to 10) using look-up table.

AREA SQUARE , CODE, READONLY

ENTRY ;Mark first instruction to execute

START

LDR R0, = TABLE1 ; Load start address of Lookup table

LDR R2, = 0X40000000

LDR R1,[R2] ; Load no whose square is to be find

MOV R1, R1, LSL#0x2 ; Generate address

ADD R0, R0, R1 ; Load address of element in Lookup table

LDR R3, [R0] ; Get square of given no in R3

NOP NOP

NOP

back B back

;Lookup table contains Squares of nos from 0 to 10 (in hex)

TABLE1	DCD 0X00000000;	SQUARE OF 0=0
	DCD 0X00000001;	SQUARE OF 1=1
	DCD 0X00000004;	SQUARE OF 2=4
	DCD 0X00000009;	SQUARE OF 3=9
	DCD 0X00000010;	SQUARE OF 4=16
	DCD 0X00000019;	SQUARE OF 5=25

DCD 0X00000024;

SQUARE OF 6=36

DCD 0X00000031;

SQUARE OF 7=49

DCD 0X00000040;

SQUARE OF 8=64

DCD 0X00000051;

SQUARE OF 9=81

DCD 0X00000064;

SQUARE OF 10=100

END

; Mark end of file

RESULT:

C:\Users\Guru GPI\Desktop\CS dept FSDP\lookuptable\p1.uvproj - uVision4

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Registers

Register	Value
R0	0x0000003C
R1	0x00000014
R2	0x40000000
R3	0x00000019
R4	0x00000000
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000000
R15 (PC)	0x00000024
CPSR	0x00000003

Disassembly

```
18: back B back
0x00000024: EAffffff B 0x00000024
0x00000028: 00000000 ANDEQ R0,R0,R0
0x0000002C: 00000001 ANDRO R0,R0,R1
```

square.asm

```
14: NOP
15: NOP
16: NOP
17:
18: back B back
19:
20: ;Lookup table contains Squares of nos from 0 to 10 (in hex)
21:
22: TABLE1 DCD 0X00000000; SQUARE OF 0=0
23: DCD 0X00000001; SQUARE OF 1=1
24: DCD 0X00000004; SQUARE OF 2=4
25: DCD 0X00000009; SQUARE OF 3=9
26: DCD 0X00000010; SQUARE OF 4=16
```

Command

Running with Code Size Limit: 32K
Load "C:\Users\Guru GPI\Desktop\CS dept FSDP\lookuptable\p1.axf"

*** Restricted Version with 32768 Byte Code Size Limit
*** Currently used: 88 Bytes (0%)

Memory 1

Address: 0x40000000

0x40000000:	05 00
0x40000017:	00 00
0x4000002E:	00 00
0x40000045:	00 00
0x4000005C:	00 00
0x40000073:	00 00
0x4000008A:	00 00
0x400000A1:	00 00
0x400000B8:	00 00
0x400000CF:	00 00

ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE

Stop code execution Real-Time Agent: Target Stopped Simulation t1: 1.79441075 sec L:18 C:1 CAP NUM SCRL OVR: R/W

Windows taskbar: 12:29 AM

Write a program to find the largest/smallest number in an array of 32 numbers.

NOP

NOP NOP back B

back

; ARRAY OF 32 BIT NUMBERS (N=7)

VALUE1

DCD 0X44444444 ;

DCD 0X22222222 ; DCD 0X11111111 ; DCD 0X33333333 ; DCD

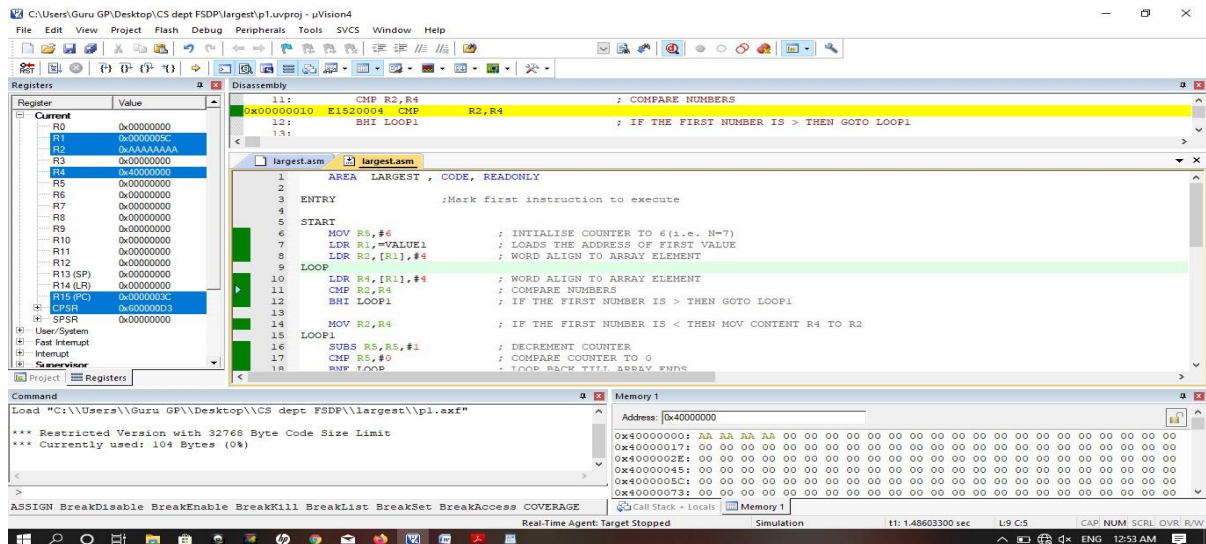
0XAAAAAAAA ; DCD 0X88888888 ; DCD 0X99999999 ;

AREA DATA2,DATA,READWRITE T O STORE RESULT IN GIVEN ADDRESS

RESULT DCD 0X0

END ; Mark end of file

RESULT



Program 7

Display “Hello World” message using Internal UART.

```
#include<lpc214x.h> void uart_init(void);
unsigned int delay;

unsigned char *ptr;

unsigned char arr[]="HELLO WORLD\r";

int main()
{
    while(1)
    {
        uart_init();          ptr =
arr;        while(*ptr!='\0')
        {
            U0THR=*ptr++;
            while(!(U0LSR & 0x40)== 0x40);
            for(delay=0;delay<=600;delay++);
        }
        for(delay=0;delay<=60000;delay++);
    }
}

void uart_init(void)
{
    PINSEL0=0X00000005;        //select TXD0 and RXD0 lines
```

```

        UOLCR = 0X00000083;                //enable baud rate divisor loading
        UODLM = 0X00;                      //select the data format
        UODLL = 0x13;                      //select baud rate 9600 bps
        UOLCR = 0X00000003;

}

```

Result: Hello World will be displayed on terminal.

Program 8

Interface a Stepper motor and rotate it in clockwise and anticlockwise direction.

Note: Stepper motor is connected to port pins P0.12 to P0.15.

```

#include <LPC21xx.H> void clock_wise(void);
void anti_clock_wise(void); unsigned long int
var1,var2;

unsigned int i=0,j=0,k=0;

int main(void)
{
        PINSEL0 = 0x00FFFFFF;                //P0.12 to P0.15 GPIO
        IOODIR = 0x0000F000;                //P0.12 to P0.15 output

        while(1)
        {
                for(j=0;j<50;j++)

                clock_wise();

                for(k=0;k<65000;k++);

```

```

        for(j=0;j<50;j++)
        anti_clock_wise();

        for(k=0;k<65000;k++);
    }
    // End of while(1)

}
// End of main

void clock_wise(void)
{
    var1 = 0x00000800;          //For Clockwise    for(i=0;i<=3;i++)    // for A B C D
Stepping
    {
        var1 = var1<<1;        //For Clockwise          IOOPIN = var1;
        for(k=0;k<3000;k++);    //for step speed variation

    }
}

void anti_clock_wise(void)
{
    var1 = 0x00010000;    //For Anticlockwise    for(i=0;i<=3;i++)    // for A B C D
Stepping
    {
        var1 = var1>>1;        //For Anticlockwise          IOOPIN = var1;
        for(k=0;k<3000;k++);    //for step speed variation

    }
}

```

Result: Stepper Motor rotates 360° in Clockwise and after a small delay rotates in 360° in Anticlockwise direction.

Program 9

Display the Hex digits 0 to F on a 7-segment LED interface, with an appropriate delay in between.

Note: Seven segment LEDS are connected to port pins P0.16 to P0.23 and select lines of 4 seven segment LEDS are connected to port pins P0.28 to P0.31.

```
#include <LPC21xx.h>
```

```
unsigned int delay, count=0, Switchcount=0; unsigned int Disp[16]={0x003F0000, 0x00060000, 0x005B0000,
0x004F0000, 0x00660000,0x006D0000, 0x007D0000, 0x00070000, 0x007F0000, 0x006F0000,
0x00770000,0x007C0000, 0x00390000, 0x005E0000, 0x00790000, 0x00710000
};
```

```
#define ALLDISP 0xF0000000          //Select all display int main (void)
```

```
{
    PINSELO = 0x00000000;
    PINSEL1 = 0x00000000;
    IOODIR = 0xF0FF0000;

    while(1)
    {
        IOOSET = ALLDISP;      IOOCLR = 0x00FF0000;
        for(delay=0;delay<100;delay++);

        IOOSET = Disp[Switchcount]; // display the values 0 to F one after the other
        for(delay=0;delay<1000000;delay++);  Switchcount++;

        if(Switchcount == 16)                // after F go back to 0
        {
            Switchcount = 0;
        }
    }
}
```

Result: Numbers 0000, 1111, 2222 ,FFFF is displayed on Seven segment display with a delay in between.

Program 10

Interface a 4x4 keyboard and display the key code on an LCD.

```
#include<lpc21xx.h> #include<stdio.h>
void lcd_init(void); void clr_disp(void); void
lcd_com(void); void lcd_data(void); void
wr_cn(void); void wr_dn(void); void
scan(void); void get_key(void); void
display(void); void delay(unsigned int);

void init_port(void);
unsigned long int scan_code[16]=
{0x00EE0000,0x00ED0000,0x00EB0000,0x00E70000,
    0x00DE0000,0x00DD0000,0x00DB0000,0x00D70000,
    0x00BE0000,0x00BD0000,0x00BB0000,0x00B70000,
0x007E0000,0x007D0000,0x007B0000,0x00770000};
unsigned char ASCII_CODE[16]= {'0','1','2','3',
    '4','5','6','7',
    '8','9','A','B',
    'C','D','E','F'};

unsigned char row,col;      unsigned char temp,flag,i,result,temp1; unsigned int r,r1; unsigned long int
var,var1,var2,res1,temp2,temp3,temp4; unsigned char *ptr,disp[] = "4X4 KEYPAD"; unsigned char disp0[] = "KEYPAD
TESTING"; unsigned char disp1[] = "KEY = ";
int main()
{
    init_port();           //port intialisation
    delay(3200);           //delay
    lcd_init();            //lcd intialisation
    delay(3200);           //delay
    clr_disp(); //clear display delay(500);           //delay

    //.....LCD DISPLAY TEST.....//

    ptr = disp;
    temp1 = 0x81;          // Display starting address    lcd_com();
    delay(800);

    while(*ptr!='\0')
    {
        temp1 = *ptr;
        lcd_data();
        ptr ++;
```



```

}

//.....KEYPAD Working.....//
while(1)
{
    get_key();
    display();
}
} //end of main()

```

```

void get_key(void)        //get the key from the keyboard
{
    unsigned int i;
    flag = 0x00;
    IO1PIN=0x000f0000;
    while(1)
    {
        for(row=0X00;row<0X04;row++)        //Writing one for col's
        {
            if( row == 0X00)
            {
                temp3=0x00700000;
            }
            else if(row == 0X01)
            {
                temp3=0x00B00000;
            }
            else if(row == 0X02)
            {
                temp3=0x00D00000;
            }
            else if(row == 0X03)
            {
                temp3=0x00E00000;
            }

            var1 = temp3;
            IO1PIN = var1;        // each time var1 value is put to port1
            IO1CLR=~var1;        // Once again Conforming (clearing all other bits)        scan();
            delay(100);        //delay
        }
    }
}

```

```

        if(flag == 0xff)        break;
    } // end of for
        if(flag == 0xff)
            break;
    } // end of while

    for(i=0;i<16;i++)
    {
        if(scan_code[i] == res1)    //equate the scan_code with res1
        {
            result = ASCII_CODE[i];    //same position value of ascii code
            break;
        }
        //is assigned to result
    }
} // end of get_key();
void scan(void)
{
    unsigned long int t;
    temp2 = IO1PIN;                // status of port1
    temp2 = temp2 & 0x000F0000;    // Verifying column key    if(temp2 != 0x000F0000)    // Check for
    Key Press or Not
    {
        delay(1000);                //delay(100)//give debounce delay check again
        temp2 = IO1PIN;
        temp2 = temp2 & 0x000F0000;    //changed condition is same

        if(temp2 != 0x000F0000)    // store the value in res1
        {
            flag = 0xff;    res1 = temp2;
            t = (temp3 & 0x00F00000);    //Verfying Row Write
            res1 = res1 | t;    //final scan value is stored in res1
        } else {
            flag = 0x00;
        }
    }
} // end of scan()

void display(void)
{
    ptr = disp0;
    temp1 = 0x80;                // Display starting address of first line    lcd_com();

```

```

        while(*ptr!='\0')
    {
        temp1 = *ptr;
        lcd_data();
        ptr ++;
    }

ptr = disp1;
temp1 = 0xC0;                                // Display starting address of second line
lcd_com();

    while(*ptr!='\0')
    {
        temp1 = *ptr;
        lcd_data();
        ptr ++;
    }

temp1 = 0xC6;                                //display address for key value
    lcd_com();
temp1 = result;    lcd_data();
}

void lcd_init (void)
{

    temp = 0x30;
    wr_cn();
    delay(3200);

    temp = 0x30;
    wr_cn();

    delay(3200);

    temp = 0x30;
    wr_cn();

```

```

    delay(3200);

    temp = 0x20;
    wr_cn();

    delay(3200);

// load command for lcd function setting with lcd in 4 bit mode,
// 2 line and 5x7 matrix display

    temp = 0x28;
    lcd_com();
    delay(3200);
// load a command for display on, cursor on and blinking off
    temp1 = 0x0C;
    lcd_com();
    delay(800);

// command for cursor increment after data dump
    temp1 = 0x06; lcd_com();
    delay(800);

    temp1 = 0x80; lcd_com();
    delay(800);
}

void lcd_data(void)
{
    temp = temp1 & 0xf0; wr_dn();
    temp = temp1 & 0x0f;   temp = temp
    << 4; wr_dn();

    delay(100);
}

void wr_dn(void)                ////write data reg

```

```

{
    IOOCLR = 0x000000FC; // clear the port lines.
    IOOSET = temp;        // Assign the value to the PORT lines
    IOOSET = 0x00000004;  // set bit RS = 1      IOOSET = 0x00000008;      // E=1  delay(10);
    IOOCLR = 0x00000008;
}

void lcd_com(void)
{
    temp = temp1 & 0xf0; wr_cn(); temp = temp1 & 0x0f; temp = temp << 4; wr_cn(); delay(500);
}

void wr_cn(void)      //write command reg
{
    IOOCLR = 0x000000FC;          // clear the port lines.
    IOOSET      = temp;          // Assign the value to the PORT lines
    IOOCLR = 0x00000004;          // clear bit RS = 0
    IOOSET      = 0x00000008;     // E=1
    delay(10);
    IOOCLR = 0x00000008;
}

void clr_disp(void)
{
    // command to clear lcd display  temp1 =
    0x01;  lcd_com();

    delay(500);
}

void delay(unsigned int r1)
{
    for(r=0;r<r1;r++);
}

void init_port()
{

```

```
IO0DIR = 0x000000FC;           //configure o/p lines for lcd
IO1DIR = 0xFFFF0FFF;
}
```

Result: out of 16 keys, one of the keys is pressed and value assigned to that key is displayed on LCD

VIVA QUESTION& ANSWERS

1. What is an embedded system?

An embedded system is a special purpose computer system which is completely encapsulated by device it control. It is a programmed hardware device in which the hardware chip is programmed with specific function. It is a combination of hardware and software.

2. What are the characteristics of embedded system?

The Characteristics of the embedded systems are as follows-

Sophisticated functionality

Real time behaviour

Low manufacturing cost

Low power consumption

User friendly

Small size

3. What are the types of embedded system?

They are of 4 types

General computing

Control System

Digital Signal Processing

Communication and network

4. What is digital signal controller?

DSC is 16-bit RISC machine that combines control advantages of micro-controller and digital signal processing to produce tightly coupled single chip-single instruction stream solution for embedded system design.

5. What are the components of embedded system?

Microcontroller, microprocessor, DSC, DSP, busses, system clock, Read only Memory(ROM), RAM, Real time clock these are the components of embedded system.

6. Why we use embedded systems?

Embedded systems avoid lots of electronic components and they have rich built in functionality. They reduce the cost and maintenance cost and the probability of failure of embedded system is less so embedded system are in very much use now a days.

7. What are the languages used in embedded system?

Assembly language and C are basically used for embedded system. Java and ADA are also preferred.

8. How does combination of functions reduce memory requirement in embedded system?

By using functions the amount of code that has to be dealt with is reduced thus redundancy is eliminated for everything common in function.

9. What is the significance of watchdog timer in ES?

It is a timing device which is set to predefined time interval and some task is to be performed at that time. It is used to reset original state when an inappropriate event take place. It is usually operated by counter device.

10. What is the difference between mutexes and semaphores?

Semaphores are the synchronization tool to overcome critical section problem.

Mutex is also a tool that is used to provide deadlock free mutual exclusion. It protects access to every critical data item, if the data is locked and is in use, it either waits for the thread to finish or awakened to release the lock from its inactive state.

11. What is the difference between FIFO and the memory?

FIFO (first in first out) is a memory structure where data's can be stored and retrieved. This is a queue where memory is a storage device which can hold data's dynamically or at any desired locations and can be retrieved in any order.

12. What is an anti-aliasing filter?

Anti-aliasing filter reduces errors due to aliasing.

13. How to implement a fourth order Butter worth LP filter at 1 KHz if sampling frequency is 8 KHz?

A fourth order butter worth filter can be made as cascade of two second order LP filters with zeta of 0.924 and 0.383. One can use a bilinear transformation approach for realising second order LP filters. Using this technique described well in many texts, one can make second order LP filters and cascade them

14. Is 8085 an embedded system?

It's not an embedded system. B'coz it will be a part of an embedded system and it does not work on any software.

15.What is the role of segment register?

In the 8086 processor architecture, memory addresses are specified in two parts called the segment and the offset. Segment values are stored in the segment registers. There are four or more segment registers: Code Segment (CS) contains segment of the current instruction (IP is the offset), Stack segment (SS) contain stack of the segment (SP is the offset), DS is the segment used by default for most data operations; ES is an extra segment register.

16.What type of registers contains an INTEL CPU?

Special function registers like accumulator, program controller (PC), data pointer (DPTR), TMOD and TCON (timing registers), 3 register banks with r0 to r7, Bit addressable registers like B.

17. What is the difference between microprocessor and micro controller?

Microprocessor is managers of the resources (I/O, memory) which lie out-side of its architecture.

Micro controllers have I/O, memory etc. built into it and specifically designed for control.

18. DMA deals with which address (physical/virtual addresses)?

DMA deals with physical addresses. DMA controller is a device which directly drives the data and address bus during data transfer. So it is purely physical address.

19. What is the difference between testing and verification?

Verification is a front-end process and testing is a post silicon process. Verification is to verify the functionality of the design during the design cycle. Testing is find manufacturing faults.

Explain what is embedded system in a computer system?

An embedded system is a special purpose computer system which is completely encapsulated by device it controls. It is a programmed hardware device in which the hardware chip is programmed with specific function. It is a combination of hardware and software.

Mention what are the essential components of embedded system?

Essential components of embedded system includes

Hardware

Processor

Memory

Timers

I/O circuits

System application specific circuits

Software

It ensures the availability of System Memory

It checks the Processor Speed availability

The need to limit power lost when running the system continuously

Real Time Operating System

It runs a process as per scheduling and do the switching from one process to another

3.What are the examples of embedded system?

An example of embedded system includes ATMs, cell phones, printers, thermostats, calculators, and videogame consoles. Handheld computers or PDAs are also considered embedded devices because of the nature of their hardware design, even though they are more expandable in software terms.

4.What is embedded c?

The C standard doesn't care about embedded, but vendors of embedded systems usually

provide standalone implementations with whatever amount of libraries they're willing to provide. C is a widely used general purpose high level programming language mainly intended for system programming.

5. What is main difference between C and embedded C?

As, embedded C is generally an extension of the C language, they are more or less similar. However, some differences do exist, such as: C is generally used for desktop computers, while embedded C is for microcontroller based applications. C can use the resources of a desktop PC like memory, OS, etc.

6. What is a Watchdog Timer?

A watchdog timer is an electronic device or electronic cards that execute specific operation after certain time period if something goes wrong with an electronic system.

7. What is the need for an infinite loop in embedded systems?

Embedded systems require infinite loops for repeatedly processing or monitoring the state of the program. For instance, the case of a program state continuously being verified for any exceptional errors that might just happen during run-time such as memory outage or divide by zero, etc.

8. What is semaphore?

A semaphore is an abstract data type or variable that is used for controlling access, by multiple processes to a common resource in a concurrent system such as multiprogramming operating system. Semaphores are commonly used for two purposes

To share a common memory space

To share access to files

9. What is the ARM7TDMI?

The ARM7TDMI is a member of the Advanced RISC Machines (ARM) family of general purpose 32-bit microprocessors, which offer high performance for very low power consumption and price.

Mention the features of ARM

32-bit RISC-processor core (32-bit instructions),• 37 pieces of 32-bit integer registers (16 available),• Thumb instruction set,• Pipelined (ARM7: 3 stages),• Cached (depending on the implementation),• Von Neuman-type bus structure (ARM7), Harvard (ARM9),• Debug Interface,• Embedded ICE macrocell ,• Jazelle DBX(Direct Bytecode eXecution).

What is meant by Pipelining

Pipelining is a technique that implements a form of parallelism called instruction-level parallelism within a single **processor**.

List out the features in LPC2148 ARM microcontrollers (K1,CO2)

32-bit ARM7TDMI-S microcontroller in a tiny LQFP64 package.8 to 40 kB of on-chip static RAM and 32 to 512 kB of on-chip flash program memory. In-System/In-Application Programming (ISP/IAP) via on-chip boot-loader software. Single flash sector or full chip erase in 400 ms and programming of 256 bytes in 1 ms. Embedded ICE RT and Embedded Trace interfaces offer real-time debugging with the on-chip Real Monitor software and high speed tracing of instruction execution. USB 2.0 Full Speed compliant Device Controller with 2 kB of endpoint RAM.In addition, the LPC2146/8 provide 8 kB of onchip RAM accessible to USB by DMA.One or two (LPC2141/2 vs. LPC2144/6/8) 10-bit A/D converters provide a total of 6/14analog inputs, with conversion times as low as 2.44 ms per channel.

12.Write the Application of GPIO in LPC2148 ARM Microcontrollers

It is used to to interface the digital input device like limit switch , digital sensors, keypad etc.,

It is used to interface the digital out put devices like LCD ,buzzer and etc.,

13.What is RTC

The Real Time Clock (RTC) is a set of counters for measuring time when system power is on, and optionally when it is off.

[

14.What is the Purpose of RTC

Measures the passage of time to maintain a calendar and clock.Provides Seconds, Minutes, Hours, Day of Month, Month, Year, Day of Week, and Day of Year.

15.What is the UART COMMUNICATION?

A universal asynchronous receiver/transmitter (UART) is a microchip that performs serial-to-parallel conversion of data received from peripheral devices and parallel-to-serial conversion of data coming from the CPU for transmission to peripheral devices

16. Mention the feature of UART in LPC2148 ARM microcontrollers

16 byte Receive and Transmit FIFOs

Register locations conform to '550 industry standard.

Receiver FIFO trigger points at 1, 4, 8, and 14 bytes.

Built-in fractional baud rate generator with autobauding capabilities.

Mechanism that enables software and hardware flow control implementation.

17.Mention the feature of ADC in LPC2148 ARM microcontrollers (K1,CO3)

10 bit successive approximation analog to digital converter Measurement range 0 V to VREF (typically 3 V; not to exceed VDDA voltage level).10 bit conversion time 2.44 μ s.Burst conversion mode for single or multiple inputs.

18.Mention the feature of SPI interface

Synchronous, Serial, Full Duplex communication.

Combined SPI master and slave.

Maximum data bit rate of one eighth of the input clock rate.

8 to 16 bits per transfer

19. Write the advantages of I²c bus in LPC2148 ARM microcontrollers (K1, CO3).

The I2C (Inter-IC) bus is a bi-directional two-wire [serial bus](#) that provides a communication link between integrated circuits ([ICs](#)).

Standard I2C compliant bus interfaces that may be configured as Master, Slave, or Master/Slave.

Arbitration between simultaneously transmitting masters without corruption of serial data on the bus.

Programmable clock to allow adjustment of I2C transfer rates.

Bidirectional data transfer between masters and slaves.

List out the types data transfer instruction set in LPC2148 ARM microcontrollers

20. What is the RTOS?

A real time operating system (RTOS) is an operating system that guarantees a certain **capability** within a **specified time constraint**.

18. What is a Thread? What are the differences between process and thread?

A thread is a single sequence stream within in a process. Because threads have some of the properties of processes, they are sometimes called *lightweight processes*. Threads are popular way to improve application through parallelism. For example, in a browser, multiple tabs can be different threads. MS word uses multiple threads, one thread to format the text, other thread to process inputs, etc.

A thread has its own program counter (PC), a register set, and a stack space. Threads are not independent of one other like processes as a result threads shares with other threads their code section, data section and OS resources like open files and signals

19. What are the different scheduling algorithms

First-Come, First-Served (FCFS) Scheduling. Shortest-Job-Next (SJN) Scheduling. Priority Scheduling. Shortest Remaining Time. Round Robin(RR) Scheduling. Multiple-Level Queues Scheduling.

20.What is deadlock?

Deadlock is a situation when two or more processes wait for each other to finish and none of them ever finish. Consider an example when two trains are coming toward each other on same track and there is only one track, none of the trains can move once they are in front of each other. Similar situation occurs in operating systems when there are two or more processes hold some resources and wait for resources held by other(s).

21.List out some popular Real Time Operating Systems(RTOS) μ C/OS – II,POSIX, VxWorks, OSOpen, OS-9, pSOSsystem, RTEMS,Linux/RT-Linux, Virtuoso, Windows CE, PalmOS, QNX Neutrino,

22.What is meant by kernel?

A kernel, executive or nucleus is the smallest portion of the operating system that provides for task scheduling, dispatching, and inter task communication.

23. What is meant by task?

A task is an independent thread of execution that can compete with other concurrent tasks for processor execution time.

24.What is multitasking and its types?

Multitasking to share CPU time between two or more tasks.

☐ Pre-emptive Multitasking ☐ Non-preemptive Multitasking

25.What is meant by Context switching?

Context switching is the process of saving and restoring sufficient information for a real-time task so that it can be resumed after being interrupted.

26.List out the task states of RTOS Running , Ready, Block .

27.What is Rate monotonic Algorithm?

Given a set of periodic tasks and preemptive priority scheduling, then assigning priorities such that the tasks with shorter periods have higher priorities (rate-monotonic), yields an optimal scheduling algorithm.

28. Mention the features of µc/os-II

The Real-Time Kernel is a portable, ROMable, scalable, preemptive real-time, multitasking kernel for microprocessors and microcontrollers.

29. List out the task management function in µc/os-II.

The functions to create task, suspend and resume, and time setting and time retrieving functions

OSTaskCreate()

OSTaskSuspend()

OSTaskResume()

OSTimeSet()

OSTimeGet()

30. List out the time management function in µc/os-II

OSTimeDly()

OSTimeDlyHMSM()

OSTimeDlyResume()

31. What is Bandwidth ?

The number of bits processed by the processor in a single instruction.

32. What is Clock Speed ?

Clock speed is measured in the MHz and it determines that how many instructions a processor can processed. The speed of the microprocessor is measured in the MHz or GHz.

33. What are the features of Intel 8086 ?

Features:

Released by Intel in 1978 □ Produced from 1978 to 1990s □ A 16-bit microprocessor chip.

Max. CPU clock rate: 5 MHz to 10 MHz

Instruction set: x86-16

Package: 40 pin DIP

16-bit Arithmetic Logic Unit

16-bit data bus (8088 has 8-bit data bus)

20-bit address bus - $2^{20} = 1,048,576 = 1 \text{ meg}$ □ The address refers to a byte in memory.

In the 8088, these bytes come in on the 8-bit data bus. In the 8086, bytes at even addresses come in on the low half of the data bus (bits 0-7) and bytes at odd addresses come in on the upper half of the data bus (bits 8-15).

The 8086 can read a 16-bit word at an even address in one operation and at an odd address in two operations. The 8088 needs two operations in either case.

The least significant byte of a word on an 8086 family microprocessor is at the lower address.

34 What is Logical Address:?

A memory address on the 8086 consists of two numbers, usually written in hexadecimal and separated by a colon, representing the *segment* and the *offset*. This combination of segment and offset is referred to as a *logical address*

Logical address=segment: offset

35. What is The Effective Address:

In general, memory accesses take the form of the following example:

Mov ax, [*baseReg* + *indexReg* + *constant*]

This example copies a word sized value into the register AX.

Combined, the three parameters in brackets determine what is called the *effective address*, which is simply the offset referenced by the instruction

36. What is Physical Address?

Physical memory address pointed by SEGMENT:OFFSET pair is calculated as:

Physical address = (<Segment Addr> * 10) + <Offset Addr>

37. What are the flags in 8086?

In 8086 Carry flag, Parity flag, Auxiliary carry flag, Zero flag, Overflow flag, Trace flag, Interrupt flag, Direction flag, and Sign flag.

38. Why crystal is a preferred clock source?

Because of high stability, large Q (Quality Factor) & the frequency that doesn't drift with aging. Crystal is used as a clock source most of the times.

39. What is Tri-state logic?

Three Logic Levels are used and they are High, Low, High impedance state. The high and low are normal logic levels & high impedance state is electrical open circuit conditions. Tri-state logic has a third line called enable line.

40. What happens when HLT instruction is executed in processor?

The Micro Processor enters into Halt-State and the buses are tri-stated.

41. What is Program counter?

Program counter holds the address of either the first byte of the next instruction to be fetched for execution or the address of the next byte of a multi byte instruction, which has not been completely fetched. In both the cases it gets incremented automatically one by one as the instruction bytes get fetched. Also Program register keeps the address of the next instruction.

42.What is 1st / 2nd / 3rd / 4th generation processor?

The processor made of PMOS / NMOS / HMOS / HCMOS technology is called 1st / 2nd / 3rd / 4th generation processor, and it is made up of 4 / 8 / 16 / 32 bits.

43.Name the processor lines of two major manufacturers?

High-end: Intel - Pentium (II, III, 4), AMD - Athlon. Low-end: Intel - Celeron, AMD - Duron. 64-bit: Intel - Itanium 2, AMD - Opteron.

44.How many bit combinations are there in a byte?

Byte contains 8 combinations of bits.

45.Have you studied buses? What types?

There are three types of buses.

Address bus: This is used to carry the Address to the memory to fetch either Instruction or Data.

Data bus : This is used to carry the Data from the memory.

Control bus : This is used to carry the Control signals like RD/WR, Select etc.

46.What is the Maximum clock frequency in 8086?

5 Mhz is the Maximum clock frequency in 8086.

47.What is meant by Maskable interrupts?

An interrupt that can be turned off by the programmer is known as Maskable interrupt.

48.What is non-Maskable interrupts?

An interrupt which can be never be turned off (ie. disabled) is known as Non-Maskable interrupt

49.What are the different functional units in 8086?

Bus Interface Unit and Execution unit, are the two different functional units in 8086.

50.What are the various segment registers in 8086?

Code, Data, Stack, Extra Segment registers in 8086.

51.What does EU do?

Execution Unit receives program instruction codes and data from BIU, executes these instructions and store the result in general registers.

52.Which Stack is used in 8086? k is used in 8086?

FIFO (First In First Out) stack is used in 8086.In this type of Stack the first stored information is retrieved first.

53.What is SIM and RIM instructions?

SIM is Set Interrupt Mask. Used to mask the hardware interrupts.

RIM is Read Interrupt Mask. Used to check whether the interrupt is Masked or not.

54.What are the different types of Addressing Modes?

A:- There are 12 different types of Addressing Modes.They are:-

<1> Immediate:-The Immediate data is a part of instruction, and appears in the form of successive bytes.

<2> Direct:-A 16-bit memory address(offset) is directly specified in the instruction as a part of it.

<3> Register:-Data is stored in a register and it is referred using the particular register (except IP).

<4> Register Indirect:-The address of the memory location which contains data or operand is determined in an indirect way.

<5> Indexed:-offset of the operand is stored in one of the index registers.

<6> Register Relative:-The data is available at an effective address formed by adding an 8-bit or 16-bit displacement with the content of any one of the registers BX,BP,SI and DI in the default (either DS or ES) segment.

<7> Based Indexed:-The effective address of the data is formed,in this addressing mode,by adding content of a base register to the content of an index register.

<8> Relative Based Indexed:- The effective address is formed by adding an 8 or 16-bit displacement with the sum of contents of any one of the base registers and any one of the index registers,in the default segment.

<9> Intrasegment Direct Mode:-In this mode,the address to which the control is to be transferred lies in the segment in which the control transfer instruction lies and appears directly in the instruction as an immediate displacement value.

<10> Intrasegment Indirect Mode:-In this mode,the displacement to which the control is to be transferred,is in the same segment in which the control transfer instruction lies,but it is passed to the instruction indirectly.

<11> Intersegment Direct:-In this mode,the address to which the control is to be transferred is in a different segment.

<12> Intersegment Indirect:-In this mode,the address to which the control is to be transferred lies in a different segment and it is passed to the instruction indirectly sequentially.

55.What are the General Data Registers & their uses?

A:- The Registers AX,BX,CX,DX are the general Purpose 16-bit registers.AX register as 16-bit accumulator.BX register is used as an offset Storage.CX register is used as default or implied counter.Dx register is used as an implicit operand or destination in case of a few instructions.

56.What are Segment Registers & their uses?

A:-There are 4 Segment Registers Code Segment(CS),Data Segment(DS),Extra Segment(ES) & Stack Segment(SS) registers.CS is used for addressing memory locationin code.DS is used to points the data.ES refers to a segment which is essentially in another data segment.SS is used fopr addressing stack segment of memory.

57.What are Flag registers?

A:-Divided into 2 parts:-Condition code or status flags and machine control flags.

S-Sign Flag:-Is to set when the result of any computation is negative.

Z-Zero Flag:-Is to set if the result of the computation or comparision performed by the previous instruction is zero.

C-Carry Flag:-Is set when there is carry out of MSB in case of addition or a borrow in case of subtraction.

T-Trap Flag:-Is set,the processor enters the single step execution mode.

I-Interrupt Flag:-Is set,the maskable interrupts are recognised by the CPU.

D-Direction Flag:-Is set for autoincrementing or autodecrementing mode in string manipulation instructions.

AC-Auxiliary Carry Flag:-Is set if there is a carry from the lowest nibble during addition or borrow for the lowest nibble.

O-Overflow Flag:-Is setif the result of a signed operation is large enough to be accommodated in a destination register.

58.What does the 8086 Architecture contain?

A:-The complete architecture of 8086 can be divided into 2types :-Bus Interface Unit(BIU) & Execution Unit.

The BIU contains the circuit for physical address calculations and a precoding instruction byte queue & it makes the bus signals available for external interfacing of the devices.

The EU contains the register set of 8086 except segment registers and IP.It has a 16-bit ALU,able to perform arithmetic and Logic operations.

59. What are different parts for 8086 architectures?

A:- The complete architecture of 8086 can be divided into 2types :-Bus Interface Unit(BIU) & Execution Unit.

The BIU contains the circuit for physical address calculations and a precoding instruction byte queue & it makes the bus signals available for external interfacing of the devices.

The EU contains the register set of 8086 except segment registers and IP.It has a 16-bit ALU,able to perform arithmetic and Logic operations.

60..What is an Interrupts

Def:- An interrupt operation suspends execution of a program so that the system can take special action.The interrupt routine executes and normally returns control to the interrupted procedure, which then resumes execution.BIOS handles Int 00H-1FH, whereas DOS handles INT 20H-3FH.

61. What is an Opcode?

A:-The part of the instruction that specifies the operation to be performed is called the Operation code or Op code.

62.What is an Operand?

A:-The data on which the operation is to be performed is called as an Operand.

63.Explain the difference between a JMP and CALL instruction?

A:-A JMP instruction permanently changes the program counter.

A CALL instruction leaves information on the stack so that the original program execution sequence can be resumed.

64.What is meant by Polling?

A:- Polling or device Polling is a process which identifies the device that has interrupted the microprocessor.

65.What is meant by Interrupt?

A:-Interrupt is an external signal that causes a microprocessor to jump to a specific subroutine.

66.What is an Instruction?

A:-An instruction is a binary pattern entered through an input device to command the microprocessor to perform that specific function.

67.What is Microcontroller and Microcomputer?

A:- Microcontroller is a device that includes microprocessor:memory and I/O signal lines on a single chip,fabricated using VLSI technology.

Microcomputer is a computer that is designed using microprocessor as its CPU.It includes microprocessor,memory and I/O.

68.What is Assembler?

A:-The assembler translates the assembly language program text which is given as input to the assembler to their binary equivalents known as object code.

The time required to translate the assembly code to object code is called access time. The assembler checks for syntax errors & displays them before giving the object code.

69. Define Variable?

A:- A Variable is an identifier that is associated with the first byte of data item.

70. Explain Dup?

A:- The DUP directive can be used to initialize several locations & to assign values to these locations.

71. Define Pipelining?

A:- In 8086, to speed up the execution program, the instructions fetching and execution of instructions are overlapped each other. This is known as Pipelining.

72. What is the use of HLDA?

A:- HLDA is the acknowledgment signal for HOLD. It indicates whether the HOLD signal is received or not. HOLD and HLDA are used as the control signals for DMA operations.

73. Explain about "LEA"?

A:- LEA (Load Effective Address) is used for initializing a register with an offset address.

A common use for LEA is to initialize an offset in BX, DI or SI for indexing an address in memory. An equivalent operation to LEA is MOV with the OFFSET operator, which generates slightly shorter machine code.

74. What is a Microprocessor?

Microprocessor is a CPU fabricated on a single chip, program-controlled device, which fetches the instructions from memory, decodes and executes the instructions.

75. What is Instruction Set?

It is the set of the instructions that the Microprocessor can execute.